

PROJECT REPORT

AIR QUALITY INDEX (AQI) PREDICTION SYSTEM

SUMMITTED BY

FATIMA RAFAQAT

Table of content

1. Introduction	3
2. Problem Statement	3
3. Data Collection	3
4. Explratory Data Analysis (EDA)	4
4.1 Dataset Overview	4
4.2 Missing Values Analysis	4
4.3 Statistical Summary & Distribution	4
4.4 Outlier Detection	4
4.5 Feature Correlation Analysis	5
5. Data Preprocessing	5
6. Feature Engineering	6
6.1 Temporal Features –	6
6.2 AQI Computation from Pollutants –	6
6.3 Rate of Change Feature –	6
7. Model Training & Evaluation	7
7.1 Linear Regression	7
7.2 Random Forest Regressor	7
7.3 XGBoost Multi-Output Regressor	7
7.4 Multi-Output Regressor with Extended Features & Hyperparameter Tuning	8
8. Project Workflow	8
8.1 Data Collection (Hourly)	8
8.2 Daily Preprocessing & Feature Engineering	8
8.3 Model Training & Prediction (Daily)	9
8.4 Result Serving via API & Dashboard	9
9. Conclusion and Model Performance	9
Reference	10

1. Introduction

Air pollution is one of the most pressing environmental challenges of our time, directly impacting human health, climate, and ecosystems. Predicting the Air Quality Index (AQI) can help governments, environmental agencies, and citizens take precautionary measures before air quality reaches hazardous levels.

The aim of this project was to develop a machine learning–based AQI prediction system using historical air quality, stored and managed on **Hopsworks**. By training multiple models and comparing their performance, we selected the best-performing one.

2. Problem Statement

The challenge was to **predict the AQI for the next three days** for a given location, based on historical AQI and related features like **SO₂, NO₃, NO₂, NH₃ etc**

The system needed to:

- Pull data from Hopsworks
- Preprocess and prepare it for AQI prediction
- Train multiple models
- Compare model performance
- Output predictions for the next 3 days

3. Data Collection

The data for this project was collected using the OpenWeather API, which provides real-time and forecasted air quality measurements. To automate the process, I implemented a GitHub Actions workflow that runs hourly, ensuring the dataset remains consistently updated without manual intervention. Each scheduled run sends a request to the OpenWeather API and retrieves relevant atmospheric pollutant concentrations for the targeted location.

The key features collected include:

- SO₂ (Sulfur Dioxide) – an important pollutant that affects air quality and health.
- NO (Nitric Oxide) – a precursor to nitrogen dioxide and ozone formation.
- O₃ (Ozone) – a secondary pollutant with a strong impact on respiratory health.
- NO₂ (Nitrogen Dioxide) – a significant traffic-related pollutant.
- PM_{2.5} (Particulate Matter $\leq 2.5\mu\text{m}$) – fine inhalable particles that can penetrate deep into the lungs and bloodstream.

- PM_{10} (Particulate Matter $\leq 10\mu\text{m}$) – larger inhalable particles that can cause respiratory irritation and health issues.
- NH_3 (Ammonia) – contributes to particulate matter formation and affects AQI.

Once retrieved, this data is stored directly into a Hopsworks Feature Store, which acts as a centralized repository for managing and versioning the features. This approach ensures that the dataset is both historically traceable and ready for model training and prediction workflows. By using this automated pipeline, the data collection process remains scalable, consistent, and reliable for long-term AQI prediction tasks.

4. Explratory Data Analysis (EDA)

To better understand the structure, distribution, and relationships within the collected AQI dataset, an **Exploratory Data Analysis (EDA)** was performed. This step was crucial for identifying patterns, anomalies, and correlations that could impact model performance.

4.1 Dataset Overview

The dataset contained time-series air quality readings obtained from the **OpenWeather API**, enriched with pollutant concentration features such as **$\text{PM}_{2.5}$** , **PM_{10}** , **CO**, **NO_2** , **SO_2** , **O_3** , and **NO**. It also included timestamps to track the temporal evolution of air quality. An initial inspection confirmed that all expected features were present and correctly typed.

4.2 Missing Values Analysis

A heatmap of missing values revealed that the dataset was largely complete, with no significant gaps in pollutant concentration records. This ensured that minimal data imputation was needed, thereby reducing the risk of introducing bias into the training process.

4.3 Statistical Summary & Distribution

Summary statistics provided insights into central tendencies and variability for each pollutant. Distribution analysis highlighted that **NO**, **SO_2** , and **NH_3** exhibited **right-skewed** distributions, meaning their values were concentrated towards the lower range but had long tails extending to higher values. Such skewness can potentially impact model training.

4.4 Outlier Detection

Boxplots revealed notable **outliers in NO and SO_2** concentrations. These anomalies could represent either actual pollution spikes or sensor/API inconsistencies.

4.5 Feature Correlation Analysis

A correlation heatmap showed strong relationships between certain pollutant pairs, such as $\text{PM}_{2.5}$ and PM_{10} , indicating they often vary together. Other pollutants like O_3 and CO displayed weaker correlations, suggesting they may provide complementary information for predictive modeling.

The EDA results helped shape feature engineering and model selection strategies, ensuring that the dataset was well-understood before moving into preprocessing and machine learning stages.

5. Data Preprocessing

The preprocessing phase was directly informed by the findings from the Exploratory Data Analysis (EDA). During EDA, it was observed that certain variables, such as **NO** and **SO₂**, were highly right-skewed and contained extreme outliers. Additionally, missing values were minimal, and most features were numeric but varied widely in scale. These insights guided the selection of preprocessing techniques to ensure the dataset was both clean and well-prepared for machine learning models.

The first step was **removing exact duplicate rows** (excluding timestamps). This was done to maintain data integrity and prevent repeated observations from biasing the model.

Next, given the presence of extreme outliers in **NO** and **SO₂**, a **winsorization** approach was applied to cap these variables at the 1st and 99th percentiles. I chose **winsorization** for handling extreme outliers in **NO** and **SO₂** because it strikes the perfect balance between data integrity and model robustness. Outright removal of outlier rows would mean losing potentially valuable and informative observations that could capture rare but important pollution events—critical in an AQI prediction context. By capping values at the 1st and 99th percentiles, I preserved the overall distribution and trends in the dataset while minimizing the distortion that extreme values could cause in model training, ensuring both stability and realism in predictions.

Log transformations were applied to skewed variables (**NO**, **SO₂**, and **NH₃**) to reduce skewness and bring their distributions closer to normality. This step is particularly beneficial for algorithms sensitive to non-normal distributions, improving stability and performance.

Following transformation, **MinMax scaling** was applied to all relevant numerical features, including pollutants, log-transformed variables, and temporal features (**hour**, **day**, **month**, and **AQI change rate**). This ensured all inputs had comparable ranges (0–1).

We intentionally **excluded PM_{2.5}** and **PM₁₀** from the training features to avoid data leakage, as these pollutants are directly used in AQI calculation. Including them would allow the model to “cheat” by using the exact values that determine AQI,

rather than learning meaningful patterns from other environmental variables. This ensures that the model's predictions reflect genuine understanding of contributing factors, making it more robust, generalizable, and reliable for real-world scenarios.

By aligning preprocessing decisions with the patterns and challenges uncovered during EDA, the dataset was transformed into a structured, consistent, and model-ready format—significantly increasing the likelihood of robust and reliable predictions.

6. Feature Engineering

The feature engineering process aimed to create meaningful, predictive variables from raw AQI and weather data stored in the Hopsworks Feature Store. The steps included:

6.1 Temporal Features –

The `timestamp_str` column was converted to a datetime object, and three temporal components were extracted:

Hour → to capture diurnal pollution patterns (e.g., rush hour spikes).

Day → to identify short-term variations and day-level events.

Month → to capture seasonal effects such as winter smog.

6.2 AQI Computation from Pollutants –

Using US EPA AQI breakpoints[\[1\]](#), pollutant concentrations (PM2.5, PM10, CO, NO₂, SO₂, O₃) were mapped to their respective AQI values by using formula[\[2\]](#). The highest AQI among all pollutants for a given timestamp was selected as the `calculated_aqi`, representing the final AQI for that record.

6.3 Rate of Change Feature –

To capture short-term fluctuations, an `aqi_change_rate` was computed as the first-order difference of the `calculated_aqi`. This allows models to learn from sudden spikes or drops in air quality.

7. Model Training & Evaluation

This section should present each model you implemented, how it was trained, assessed, and compared. Here's how we can structure it:

7.1 Linear Regression

Workflow: Loaded preprocessed data from Hopsworks, split into train-test sets, trained a simple linear regression model to predict AQI using scaled features.

Key Metrics: MSE, MAE, and R^2 .

Performance Insight: Provides a solid baseline and interpretable benchmark for comparison with more complex models.

7.2 Random Forest Regressor

Workflow: Trained on all relevant features (excluding timestamp) from Hopsworks. Model handles non-linearity and feature interactions.

Key Metrics: MSE, MAE, R^2 .

Performance Insight: Often outperforms linear regression in complex, non-linear relationships.

7.3 XGBoost Multi-Output Regressor

Workflow: Used lag features (AQI of previous 1–3 days) and other scaled/environmental features to train a multi-output model predicting AQI for the next three days simultaneously.

Key Metrics: R^2 and/or MAE for each horizon (Day+1, Day+2, Day+3).

Performance Insight: Demonstrates the ability to forecast multiple days ahead using a strong ensemble model.

7.4 Multi-Output Regressor with Extended Features & Hyperparameter Tuning

Workflow: Added rolling statistics (7-day/14-day mean and standard deviation of AQI), cyclical encodings for time, performed hyperparameter tuning via randomized search with time-series cross-validation, and trained a final multi-output XGBoost model.

Key Metrics: Best cross-validated R^2 , final train/test R^2 for all horizons.

Performance Insight: Offers optimized performance for multi-day forecasting and shows the benefit of engineered time-series features and tuning.

8. Project Workflow

The overall workflow of the AQI Prediction System was carefully designed to ensure smooth data collection, transformation, modeling, and result visualization. The project follows a fully automated pipeline that leverages **OpenWeather API**, **Hopsworks Feature Store**, and **FastAPI** for dashboard deployment. The steps are described below:

8.1 Data Collection (Hourly)

- Raw AQI data for Islamabad is collected directly from the OpenWeather API every hour.
- This data includes pollutant concentrations (e.g., NO₂, SO₂, CO, O₃, NH₃, PM_{2.5}, PM₁₀) and related atmospheric parameters.
- The collected data is stored in its raw form in the Hopsworks Feature Store. This step ensures that no information is lost and historical data is consistently maintained for analysis and training.

8.2 Daily Preprocessing & Feature Engineering

- A separate script runs once daily to fetch the latest accumulated raw data from the feature store.
- Preprocessing steps include handling duplicates, applying outlier treatment (winsorization), scaling, and dropping direct AQI-defining variables (PM_{2.5}, PM₁₀) to avoid data leakage.

- Feature engineering is then applied to create new informative features such as log-transformed pollutant values, temporal features (hour, day, month), AQI change rate, and lag variables.
- The processed dataset is stored back in the Hopsworks Feature Store as a new version, ensuring proper lineage and reproducibility.

8.3 Model Training & Prediction (Daily)

- The cleaned and feature-engineered dataset is used to train multiple models including Linear Regression, Random Forest, XGBoost, MultiOutputRegressor.
- The best-performing models are used to generate multi-step forecasts of AQI for the next three days.
- These predictions are stored in Hopsworks, alongside timestamps, for later retrieval and comparison.

8.4 Result Serving via API & Dashboard

- A FastAPI service was developed to fetch prediction results directly from the feature store.
- The API provides endpoints that supply the most recent AQI forecasts.
- A dashboard connected to the API visualizes the predicted AQI trends for the next three days, allowing users to easily interpret upcoming air quality levels.

This automated pipeline ensures that the system continuously ingests new data, processes it, retrains models daily, and delivers up-to-date AQI forecasts with minimal manual intervention.

9. Conclusion and Model Performance

The AQI Prediction System successfully automated the end-to-end pipeline for data collection, preprocessing, feature engineering, model training, and prediction serving. By leveraging **Hopsworks Feature Store** for data management and **FastAPI** for result delivery, the project demonstrated how real-world environmental monitoring can be efficiently operationalized.

During experimentation, multiple models were trained and compared:

- **Linear Regression** provided a simple baseline but struggled with capturing non-linear relationships in AQI trends with R^2 between 80 to 90% .
- **Random Forest** offered improvements by handling complex pollutant interactions with R^2 above 90% .
- **XGBoost** emerged as one of the strongest performers, efficiently capturing non-linear dependencies with R^2 above 95% .
- **Multi-Output Regressor** extended traditional models to simultaneously predict AQI values across multiple future time horizons (e.g., the next 3 days), making it highly suitable for multi-step forecasting tasks with R^2 above 95% and also have a better R^2 for testing as well.

The evaluation revealed that **XGBoost and Multi-Output Regressor consistently outperformed other models**, achieving the lowest error rates and the most reliable 3-day AQI forecasts. These models demonstrated the ability to balance short-term variability with multi-day trend prediction, making them ideal candidates for deployment.

Overall, the project proved that machine learning models, when combined with a structured data pipeline, can provide timely and accurate air quality forecasts. This predictive capability can support policymakers, environmental agencies, and citizens in making informed decisions to mitigate the adverse effects of air pollution.

Reference

1. https://en.wikipedia.org/wiki/Air_quality_index
2. [Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index \(AQI\)](#)